LatentLearn 🧠🌲

A Spatial, Non-Linear Learning Space for Divergent Minds

Tech Stack: Next.js / FastAPI / Stateful LangGraph / OpenRouter / DeepSeek

Target: 2026 AI Agent Innovation Pitch & Demo

01. The "Linear Trap" of Traditional AI Interaction

Waterfall chat threads (like standard ChatGPT) create severe cognitive bottlenecks for associative learners:

🍂 1. Context Pollution

When exploring tangents or details, the conversational history becomes cluttered with secondary, off–topic details.

Consequence: The LLM's understanding of the primary subject degrades due to noise in the prompt window.

🌲 2. Losing the Trunk

Returning to the core subject after a deep dive requires massive scrolling and a high cognitive load.

Consequence: Frequent disruption of flow state, causing users to get lost and abandon study sessions.

02. The Spatial Paradigm: Breaking the Linear Trap

We represent the learning journey as a hierarchical "Focus Tree" on a spatial, interactive canvas

💡 Core Philosophy

Divergent minds (ADHD or associative thinkers) learn in branches, not straight lines.

We empower users to **spawn infinite sub-branches** for tangential curiosity and **collapse them cleanly** to return to the core trunk.

📍 Inline Branching

Highlight any word in AI responses to instantly spawn a localized sub-node card without cluttering history.

🛡️ Cognitive Guardrails

Background drift detection alerts you when tangents go too far, offering a 1-click return prompt.

03. Interaction Experience & UI Aesthetics

✨ 1. Premium Glassmorphic UI

Frosted glass card aesthetics (`backdrop-filter`) with responsive light/dark theme support. Micro-animations and smooth zoom transitions provide highly tactile user feedback.

📍 2. Touchpoint Context Menu

Dragging to highlight words instantly triggers a floating popover menu with `Ask`, `Explain`, and `Expand` options, anchoring curiosity contextually.

🛡️ 3. Drift Inheritance & Reset

Sub-branches of an already off-topic parent node inherit the off-topic status. Canvas alerts present a **"Refocus"** trigger to spring back to the nearest core trunk.

✅ 4. Resolution & Decluttering

Clicking "Got it " collapses and fades out tangential branches. The canvas auto-focuses back to the parent node, maintaining clean, clutter-free spatial environments.

04. Technical Architecture: Decoupled & Asynchronous

🌐 Next.js Frontend

Built with **React 18** and TypeScript 5.

• Custom **Server-Sent Events (SSE)** streaming parser.

• Local state reducers handle tree node spawning, folding, and camera pan auto-focus.

⚡ FastAPI Web Gateway

Asynchronous endpoints maximize concurrency.

• **Security Sandbox:** Cleans input via strict sanitization to block prompt injections.

• **Real-time Streams:** Establishes persistent SSE pipelines.

🌲 LangGraph Agent Engine

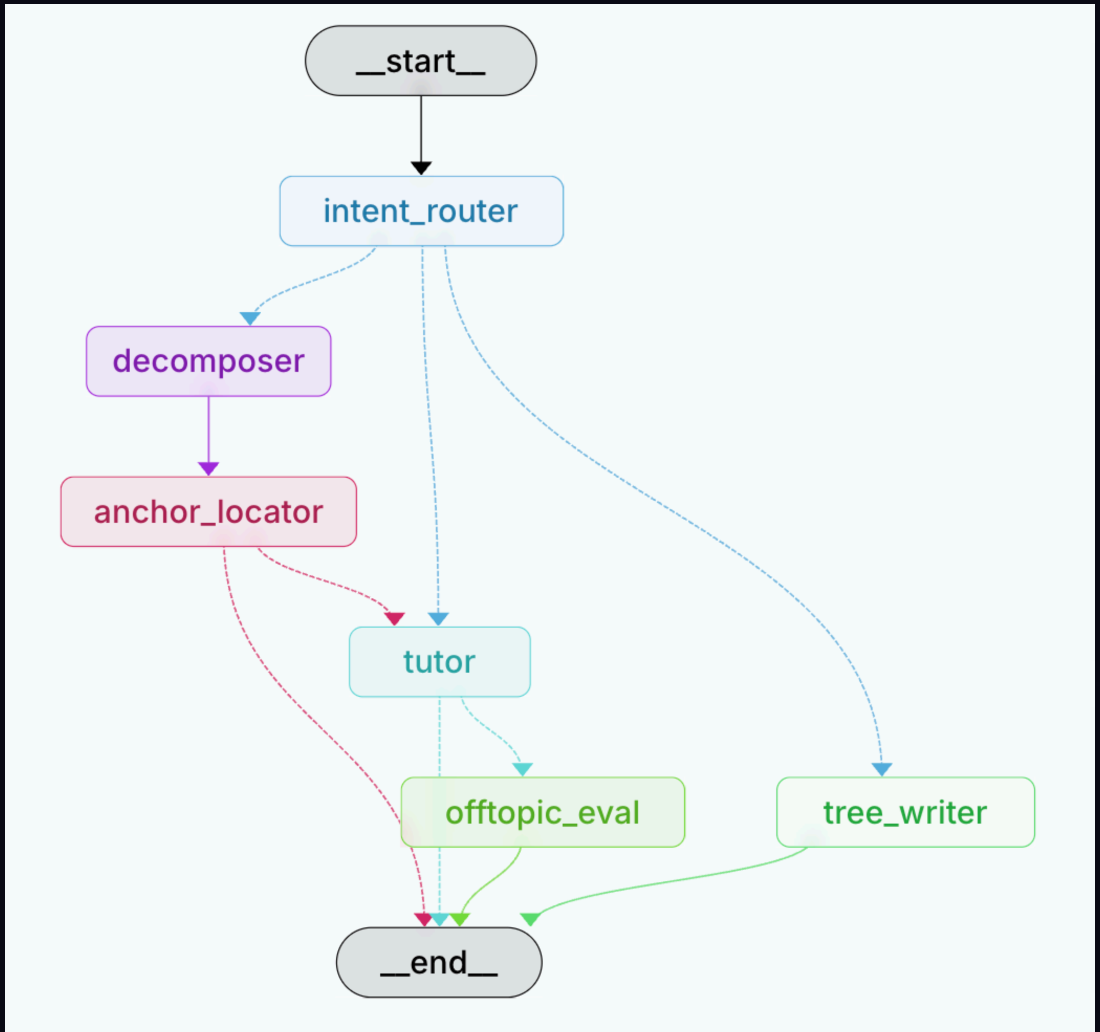
Stateful multi-agent DAG orchestration.

• **Checkpointer System:** Saves and resumes chat states on specific nodes.

• **OpenRouter Bindings:** Integrates high-performance remote LLM providers securely.

05. The Multi-Agent Brain: Stateful LangGraph DAG

Stateful nodes coordinate intent routing, structural query decomposition, context anchoring, and off-topic evaluations



1. `intent_router` (Intent Classifier)
Classifies user query to trigger the correct pipeline.

2. `decomposer` (Query Decomposer)
Splits compound queries into sequential sub-questions.

3. `tutor` & `anchor_locator`
Anchors source context and streams markdown responses.

06. Core Technical Breakthroughs

📡 1. SSE Stream Token Filtering

The Pain: When intermediate agents (decomposer, tree–writer) output structured JSONs, standard streaming dumps JSON code directly into user chat bubbles.

The Solve: Engineered an **Event–Stream Node Whitelist filter**. Uses underlying `astream_events` to intercept tokens, passing to frontend ONLY when the event is of type `on_chat_model_stream` and generated by the `tutor` node.

🌲 2. Drift Inheritance & Early Exit

Off–Topic Inheritance: Sub–questions asked on a parent node that is already off–topic automatically inherit the off–topic flag, eliminating expensive LLM validation runs.

Early Exit Optimization: Under decomposition mode, once `anchor_locator` flags sub–questions, the state machine triggers an **Early–Exit short–circuit**, bypassing the expensive tutor generation to immediately dispatch the structured QuestionPlan.

07. Developer Experience: Visual Traces & Telemetry

🖼️ 1. LangGraph Studio Local GUI

Fully integrated with **LangGraph CLI tooling**. Running `langgraph dev` launches an interactive visual workbench:

- **Live Execution Path:** Watch node highlights shift in real-time as state moves through the graph.
- **State Time-Travel:** Interactively modify, rewind, and re-run state variables to mock edge-cases and test errors.

📈 2. LangSmith Observability

Production-grade observability through **LangSmith Deep Tracing Integration**:

- **Call-level Tracing:** Inspect inputs, outputs, system prompts, latency, and exact token costs for each step.
- **Data Feedback Loops:** Group runs as good/bad to construct regression test suites, systematically optimizing prompts.

08. The Intelligent Learning Pipeline

Core Pillar	Backend Agent Logic	Intelligent Mechanism	Cognitive UX Value
1. Query Decomposition	• `decomposer` node • Structured QuestionPlan schema	• Early–Exit optimization • User intent classifier	Avoids information dump; Splits compound questions sequentially
2. Dynamic Branching	• `tree_writer` node • Anchor locator coordinate maps	• Local state reducer synchronization • Precise quote anchoring	Prevents vertical scroll chaos; Preserves exploration history
3. Off–Topic Guardrails	• `offtopic_eval` node • Stored document–summary check	• Drift status inheritance • 1–click refocusing guardrails	Detects topic drift in real–time; Gently guides focus back to core

Let thoughts grow freely, discover depth in connections.

> "The mind is not a vessel to be filled, but a fire to be kindled." —— Plutarch

Thank you for your watch!